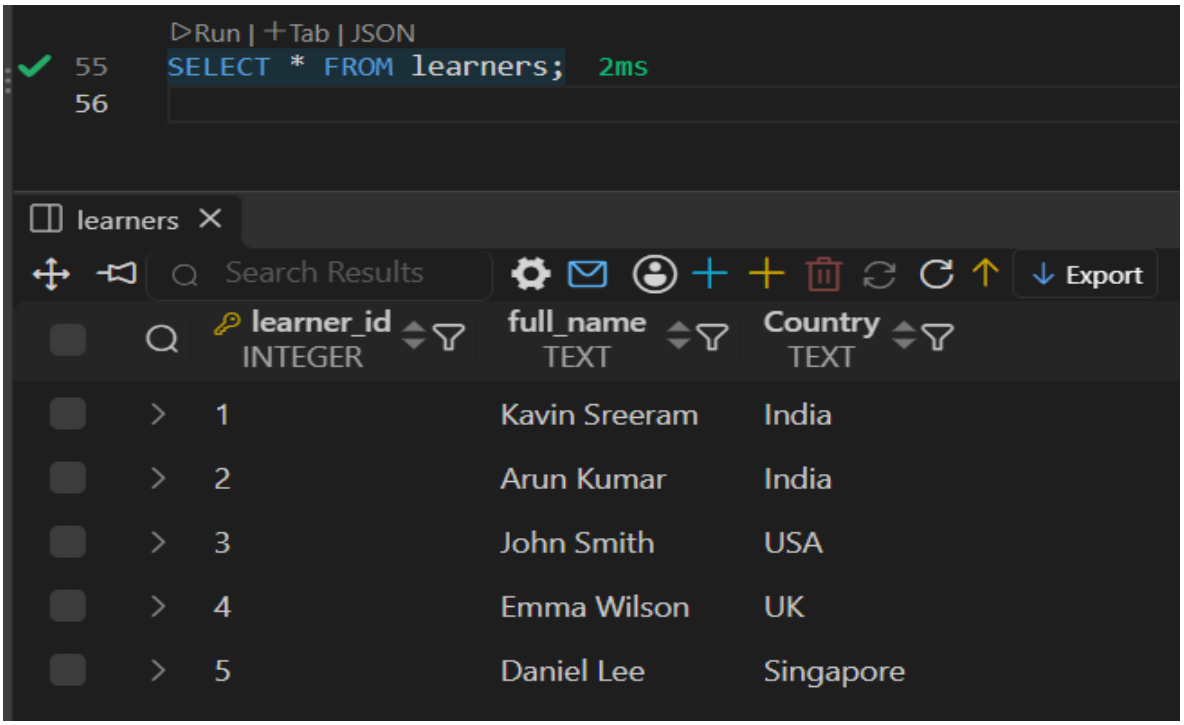
Module End Assignment Observation

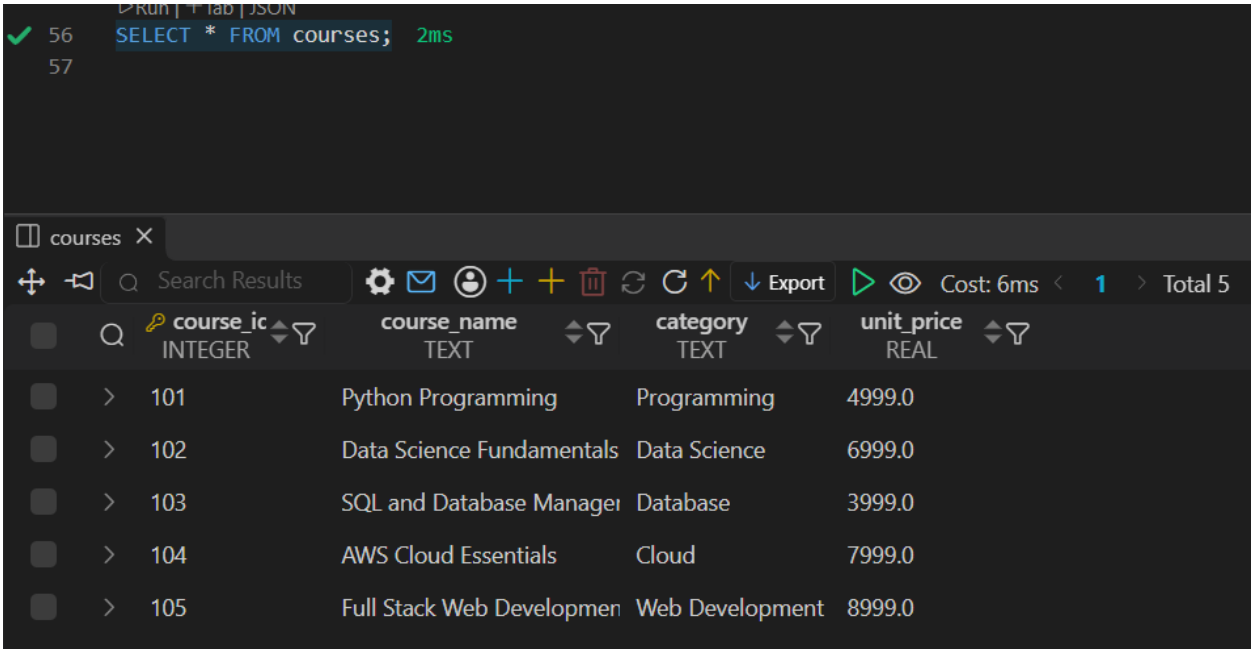
The database and required tables were created successfully, and all necessary sample data was populated without errors.

1. Create the database and schema. Populate the Schema:

Learners Table



Courses table



Purchase table

```
Run | Tab | JSON
55  SELECT * FROM purchases;  2ms
```

purchases

Export | Cost: 6ms | 1 | Total 8

purchase_id INTEGER	learner_id INTEGER	course_ic INTEGER	Quantity INTEGER	purchase_date DATE
1001	1	101	1	2026-10-01
1002	1	103	1	2026-10-03
1003	2	102	1	2026-10-05
1004	2	104	1	2026-10-07
1005	3	105	1	2026-10-10
1006	4	101	2	2026-10-12
1007	5	103	1	2026-10-15
1008	3	102	1	2026-10-18

2. Data Exploration Using Joins

2.1) Use SQL INNER JOIN, LEFT JOIN, and RIGHT JOIN: Combine learner, course, and purchase data.

```
Run | Tab | JSON | Select
55  SELECT
56      l.learner_id,
57      l.full_name AS learner_name,
58      c.course_name,
59      c.category,
60      p.Quantity AS quantity,
61      printf('%.2f', c.unit_price * p.Quantity) AS total_amount,
62      p.purchase_date
63  FROM learners l
64  INNER JOIN purchases p
65      ON l.learner_id = p.learner_id
66  INNER JOIN courses c
67      ON p.course_id = c.course_id
68  ORDER BY total_amount DESC;  256ms
```

Result(RO)

Export | Cost: 256ms | 1 | Total 8

learner_id	learner_name	course_name	category	quantity	total_amount	purchase_date
4	Emma Wilson	Python Programming	Programming	2	9998.00	2026-10-12
3	John Smith	Full Stack Web Developmen	Web Development	1	8999.00	2026-10-10
2	Arun Kumar	AWS Cloud Essentials	Cloud	1	7999.00	2026-10-07
2	Arun Kumar	Data Science Fundamentals	Data Science	1	6999.00	2026-10-05
3	John Smith	Data Science Fundamentals	Data Science	1	6999.00	2026-10-18
1	Kavin Sreeram	Python Programming	Programming	1	4999.00	2026-10-01
1	Kavin Sreeram	SQL and Database Manager	Database	1	3999.00	2026-10-03

```sql
SELECT
    l.learner_id,
    l.full_name AS learner_name,
    c.course_name,
    c.category,
    COALESCE(p.Quantity, 0) AS quantity,
    printf('%.2f', COALESCE(c.unit_price * p.Quantity, 0)) AS total_amount,
    p.purchase_date
FROM learners l
LEFT JOIN purchases p
    ON l.learner_id = p.learner_id
LEFT JOIN courses c
    ON p.course_id = c.course_id
ORDER BY total_amount DESC;  3ms
```

Result(RO)

Search Results | Export | Cost: 3ms | 1 | Total 8

learner_id	learner_name	course_name	category	quantity	total_amount	purchase_date
4	Emma Wilson	Python Programming	Programming	2	9998.00	2026-10-12
3	John Smith	Full Stack Web Developmen	Web Development	1	8999.00	2026-10-10
2	Arun Kumar	AWS Cloud Essentials	Cloud	1	7999.00	2026-10-07
2	Arun Kumar	Data Science Fundamentals	Data Science	1	6999.00	2026-10-05
3	John Smith	Data Science Fundamentals	Data Science	1	6999.00	2026-10-18
1	Kavin Sreeram	Python Programming	Programming	1	4999.00	2026-10-01
1	Kavin Sreeram	SQL and Database Manager	Database	1	3999.00	2026-10-03

```sql
SELECT
    l.learner_id,
    l.full_name AS learner_name,
    c.course_name,
    c.category,
    COALESCE(p.Quantity, 0) AS quantity,
    printf('%.2f', COALESCE(c.unit_price * p.Quantity, 0)) AS total_amount,
    p.purchase_date
FROM purchases p
RIGHT JOIN courses c
    ON p.course_id = c.course_id
LEFT JOIN learners l
    ON p.learner_id = l.learner_id
ORDER BY total_amount DESC;  3ms
```

Result(RO)

Search Results | Export | Cost: 3ms | 1 | Total 8

learner_id	learner_name	course_name	category	quantity	total_amount	purchase_date
4	Emma Wilson	Python Programming	Programming	2	9998.00	2026-10-12
3	John Smith	Full Stack Web Developmen	Web Development	1	8999.00	2026-10-10
2	Arun Kumar	AWS Cloud Essentials	Cloud	1	7999.00	2026-10-07
2	Arun Kumar	Data Science Fundamentals	Data Science	1	6999.00	2026-10-05
3	John Smith	Data Science Fundamentals	Data Science	1	6999.00	2026-10-18
1	Kavin Sreeram	Python Programming	Programming	1	4999.00	2026-10-01
1	Kavin Sreeram	SQL and Database Manager	Database	1	3999.00	2026-10-03

2.2 Display each learner's purchase details (course name, category, quantity and total amount)

```
Run | Tab | JSON | Select
100  SELECT
101      l.full_name AS learner_name,
102      c.course_name,
103      c.category,
104      p.Quantity AS quantity,
105      printf('%.2f', c.unit_price * p.Quantity) AS total_amount,
106      p.purchase_date
107  FROM learners l
108  LEFT JOIN purchases p
109      ON l.learner_id = p.learner_id
110  LEFT JOIN courses c
111      ON p.course_id = c.course_id
112  ORDER BY total_amount DESC;  2ms
```

Result(RO)

Search Results | Export | Cost: 2ms | 1 | Total 8

learner_name	course_name	category	quantity	total_amount	purchase_date
Emma Wilson	Python Programming	Programming	2	9998.00	2026-10-12
John Smith	Full Stack Web Developmen	Web Development	1	8999.00	2026-10-10
Arun Kumar	AWS Cloud Essentials	Cloud	1	7999.00	2026-10-07
Arun Kumar	Data Science Fundamentals	Data Science	1	6999.00	2026-10-05
John Smith	Data Science Fundamentals	Data Science	1	6999.00	2026-10-18
Kavin Sreeram	Python Programming	Programming	1	4999.00	2026-10-01
Kavin Sreeram	SQL and Database Manager	Database	1	3999.00	2026-10-03

3. Analytical Queries Write SQL queries to answer the following questions:

Q1. Display each learner's total spending (quantity × unit price) along with their country.

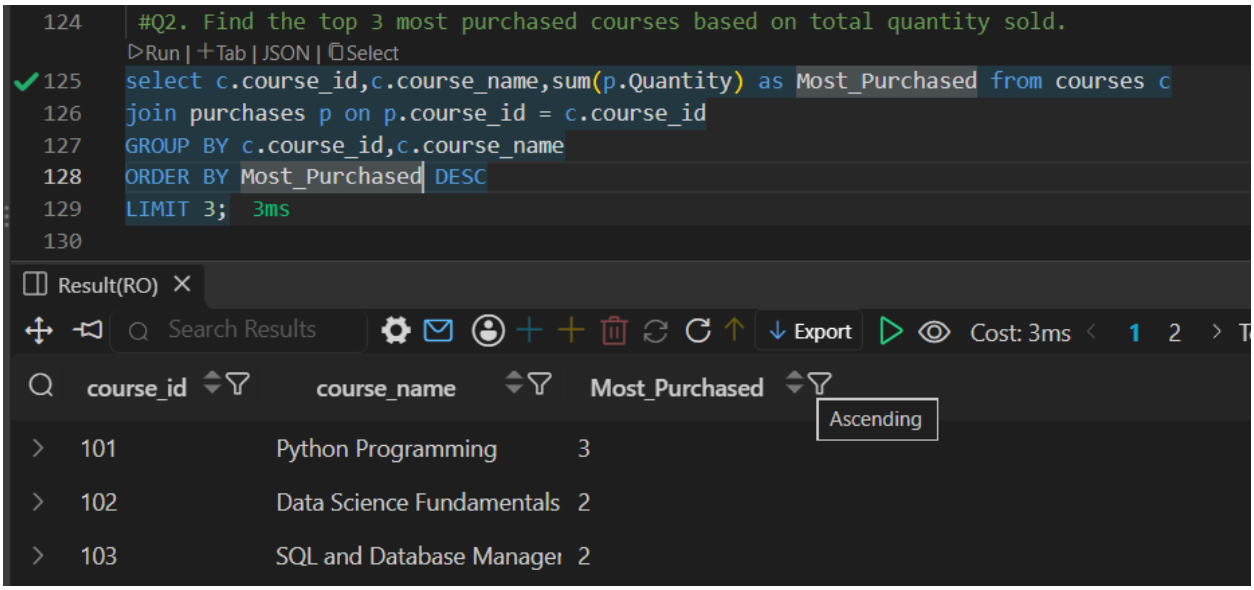
```
Run | Tab | JSON | Select
114  select l.learner_id,
115         l.full_name,
116         l.Country,
117         sum(c.unit_price * p."Quantity") as total_spending
118  from learners l
119  join purchases p on p.learner_id = l.learner_id
120  join courses c on c.course_id = p.course_id
121  GROUP BY l.learner_id,l.full_name,l.Country
122  ORDER BY total_spending desc  4ms
123
```

Result(RO)

Search Results | Export | Cost: 4ms | 1 | Total 5

learner_id	full_name	Country	total_spending
3	John Smith	USA	15998.0
2	Arun Kumar	India	14998.0
4	Emma Wilson	UK	9998.0
1	Kavin Sreeram	India	8998.0
5	Daniel Lee	Singapore	3999.0

Q2. Find the top 3 most purchased courses based on total quantity sold.



```
#Q2. Find the top 3 most purchased courses based on total quantity sold.
Run | Tab | JSON | Select
select c.course_id,c.course_name,sum(p.Quantity) as Most_Purchased from courses c
join purchases p on p.course_id = c.course_id
GROUP BY c.course_id,c.course_name
ORDER BY Most_Purchased DESC
LIMIT 3;  3ms
```

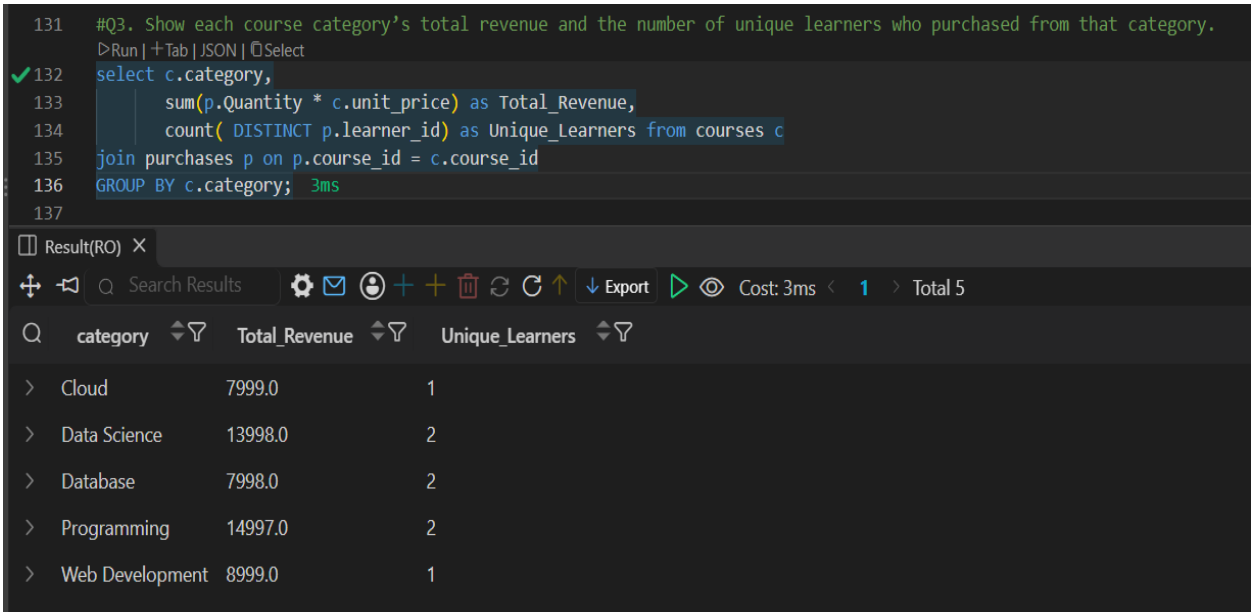
Result(RO)

Search Results | Export | Cost: 3ms | 1 2

course_id	course_name	Most_Purchased
101	Python Programming	3
102	Data Science Fundamentals	2
103	SQL and Database Manager	2

Ascending

Q3. Show each course category's total revenue and the number of unique learners who purchased from that category.



```
#Q3. Show each course category's total revenue and the number of unique learners who purchased from that category.
Run | Tab | JSON | Select
select c.category,
       sum(p.Quantity * c.unit_price) as Total_Revenue,
       count( DISTINCT p.learner_id) as Unique_Learners from courses c
join purchases p on p.course_id = c.course_id
GROUP BY c.category;  3ms
```

Result(RO)

Search Results | Export | Cost: 3ms | 1 | Total 5

category	Total_Revenue	Unique_Learners
Cloud	7999.0	1
Data Science	13998.0	2
Database	7998.0	2
Programming	14997.0	2
Web Development	8999.0	1

Q4. List all learners who have purchased courses from more than one category.

```
# List all learners who have purchased courses from more than one category.
Run | Tab | JSON | Select
select l.learner_id,l.full_name,count(DISTINCT c.category) category_cnt from purchases p
join learners l on l.learner_id = p.learner_id
join courses c on c.course_id = p.course_id
GROUP BY l.full_name,l.learner_id
HAVING category_cnt > 1;  2ms
```

Result(RO)

Search Results | Export | Cost: 2ms | 1 | Total 3

learner_id	full_name	category_cnt
2	Arun Kumar	2
3	John Smith	2
1	Kavin Sreeram	2

Q5. Identify courses that have not been purchased at all.

I did not have any unpurchased courses in the existing data, so I inserted one new course record without adding a corresponding purchase record.

```
#Q5. Identify courses that have not been purchased at all.
Run | Select
INSERT INTO courses (course_id, course_name, category, unit_price)
VALUES
(106, 'Machine Learning with Python', 'Artificial Intelligence', 9999.00);
Run | Tab | JSON
select * from courses c
where c.course_id  not in (select p.course_id from purchases p);  2ms
```

Result(RO)

Search Results | Export | Cost: 2ms | 1 | Total 1

course_id	course_name	category	unit_price
106	Machine Learning with Pyth	Artificial Intelligence	9999.0